Knowledge Representation and Problem-Solving

Tema 2: Modelare cu logica de ordinul I

Planul de învățământ Calculatoare: baza de cunoștințe, motorul de inferență și ghidul de utilizare

Master ACE Craiova

Autori: Darius Diculescu, _____

Repository: github.com/Darius762/krps_fol

Cuprins

1. Despre proiect 3
    1.1 Cerințele temei și unde sunt acoperite 3
    1.2 Baza de cunoștințe în cifre 3
    1.3 Arhitectura 3
2. Instalare și comenzi 5
    2.1 Instalare (o singură dată) 5
    2.2 Interfața în terminal (REPL) 5
    2.3 Interfața web 5
    2.4 Testele automate 6
3. Sintaxa limbajului 7
4. Faptele: materii, prerechizite, studenți 8
    4.1 Toate materiile 8
    4.2 Toate legăturile de prerechizit 10
    4.3 Studenții 12
5. Regulile 14
    5.1 necesita(M, P): prerechizitele directe și indirecte (recursiv) 14
    5.2 poate_inscrie(S, M): are TOATE prerechizitele directe 14
    5.3 ii_lipseste(S, M, P): ce îi lipsește din tot lanțul 14
    5.4 colegi(S1, S2): același an, persoane diferite 15
    5.5 restanta(S, M): a picat și nu a promovat încă 15
    5.6 eligibil_licenta(S): poate susține licența 15
    5.7 Graful de dependențe între reguli 15
6. Exemple de interogări cu rezultatele reale 16
    Fapte simple 16
    Recursivitate: necesita 17
    Toate prerechizitele: poate_inscrie 18
    Lanțuri de reguli: ii_lipseste 20
    Diferit: \= 20
    Negație ca eșec: not 21
    Interogări compuse 22
7. Trace-uri explicate pas cu pas 24
    7.1 colegi(bianca, X): backtracking 24
    7.2 eligibil_licenta(emil): două negații imbricate 24
    7.3 necesita(baze_de_date, P): recursivitate și redenumire 25
8. Cum funcționează motorul 26
    8.1 Unificarea 26
    8.2 Backward chaining (rezoluție SLD) 26
    8.3 Built-in-uri și protecții 26
9. Scenariu pentru demo-ul în fața profesorului 27
10. Întrebări posibile și răspunsuri 28

1. Despre proiect

Proiectul reprezintă planul de învățământ al specializării **Calculatoare** de la Facultatea de Automatică, Calculatoare și Electronică din Craiova în **logica de ordinul I (FOL)** și răspunde automat la întrebări despre el: la ce materii se poate înscrie un student, ce îi lipsește pentru o materie, cine poate susține licența.

Răspunsurile **nu sunt scrise** în baza de cunoștințe. Ele sunt **deduse** de un motor de inferență scris de la zero în Python, fără biblioteci de logică (fără Prolog, kanren, pyDatalog). Motorul folosește **backward chaining** (rezoluție SLD, ca Prolog), cu unificare cu occurs check, redenumirea variabilelor, operatorul `\=` și negație ca eșec (`not`).

1.1 Cerințele temei și unde sunt acoperite

Cerința	Unde se găsește
Definirea faptelor, relațiilor și regulilor în FOL	`kb/curriculum.kb`; capitolele 4 și 5 din acest document
Implementarea query answering	`fol/` (motorul), `main.py` (terminal), `web/` (interfața web)
Interogări non-triviale	capitolul 6: 40 de exemple cu rezultatele reale
Livrabile: fișierul KB, codul de interogare, exemple	repository-ul GitHub + acest document

1.2 Baza de cunoștințe în cifre

Element	Număr
Clauze în total	375
Materii (`materie/3`)	69
Materii opționale (`optional/1`)	9
Relații de prerechizit (`prerechizit/2`)	63
Studenți (`student/2`)	5
Fapte `promovat/2`	149
Reguli	8 (6 predicate principale + 1 ajutător)
Teste automate (`pytest -v`)	106

1.3 Arhitectura

```
fișier .kb ──► parser.py ──► KnowledgeBase (index pe predicat/aritate)
                                        │
?- interogare ──► parser.py ──► Engine: backward chaining ──► răspunsuri
                                        │
                                   unify.py

Interfețe:  main.py (terminal, stil Prolog)     web/ (browser)
```

Fișier	Rol
`fol/terms.py`	termenii FOL: Var (variabilă), Const (constantă), Compound (predicat cu argumente), Rule (regulă / fapt)
`fol/unify.py`	unificarea: walk, substitute, occurs check, unify; substituțiile nu se modifică niciodată pe loc
`fol/parser.py`	transformă textul .kb și interogările în obiecte; erori de sintaxă cu numărul liniei
`fol/knowledge_base.py`	încarcă fișierul și indexează clauzele după (nume, număr de argumente)

Fișier	Rol
`fol/engine.py`	motorul de inferență: backward chaining cu generatori, \=, not, limită de adâncime, trace
`main.py`	interfața în terminal (REPL)
`web/`	interfața web: server (`server.py`), API (`api.py`), exemple (`examples.py`), pagina (`static/index.html`)
`kb/curriculum.kb`	baza de cunoștințe
`tests/`	testele automate pentru fiecare modul

2. Instalare și comenzi

2.1 Instalare (o singură dată)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install pytest
```

Singura dependență este `pytest`, folosită doar pentru teste. Motorul și ambele interfețe folosesc numai biblioteca standard Python (3.11+).

2.2 Interfața în terminal (REPL)

```
python main.py                  # încarcă implicit kb/curriculum.kb
python main.py alt_fisier.kb     # altă bază de cunoștințe
```

Comandă	Ce face	Exemplu
`?- scop.`	rulează o interogare și afișează toate răspunsurile; `?-` și punctul sunt opționale	`?- poate_inscrie(cristian, M).`
`scop1, scop2`	conjuncție: toate scopurile trebuie să fie adevărate, cu aceleași variabile	`student(S, 2), poate_inscrie(S, M).`
`:help`	afișează ajutorul	`:help`
`:list`	afișează toată baza de cunoștințe	`:list`
`:list nume`	afișează doar clauzele unui predicat	`:list prerechizit`
`:trace`	pornește/oprește afișarea pașilor de inferență (Call / Exit / Fail, indentat după adâncime)	`:trace` apoi `colegi(bianca, X).`
`:quit`	ieșire (sau Ctrl+C)	`:quit`

Cum arată răspunsurile:

- cu variabile: câte o linie pentru fiecare răspuns, de exemplu `X = cristian`;
- fără variabile: `true.` dacă se poate demonstra, `false.` dacă nu;
- maximum 100 de răspunsuri (pentru interogări cu o infinitate de soluții);
- dacă se atinge limita de adâncime (200), apare un avertisment: răspunsul poate fi incomplet;
- o greșeală de sintaxă afișează eroarea cu numărul liniei, fără să oprească programul.

Exemplu de sesiune:

```
> python main.py
Am incarcat 375 clauze din kb/curriculum.kb. Scrie :help pentru ajutor.
?- eligibil_licenta(S).
S = diana
?- colegi(andrei, X).
false.
?- student(ana, .
eroare de sintaxa: linia 1: astept un termen dar am gasit '.'
?- :quit
```

2.3 Interfața web

```
python -m web                    # pornește serverul și deschide browserul
python -m web --port 8080        # alt port
python -m web --no-browser       # fără deschiderea automată a browserului
python -m web alt_fisier.kb      # altă bază de cunoștințe
```

Adresa implicită este `http://127.0.0.1:8000`. Serverul se oprește cu Ctrl+C. Pagina nu folosește resurse de pe internet, deci funcționează și offline la prezentare.

Tab	Ce conține	Cum se folosește
Interogare	câmp de interogare, bifa Trace, tabel cu răspunsuri, trace colorat, istoric	scrii interogarea și apeși Enter; click pe un predicat din listă îl inserează; click în istoric o rulează din nou
Exemple	cele 40 de exemple din capitolul 6, grupate pe categorii	„Rulează” sau „Rulează cu trace” execută exemplul în tab-ul Interogare
Graful prerechizitelor	toate cele 69 de materii, pe 8 semestre, cu săgețile de prerechizit	click pe o materie: galben = tot ce îi trebuie, mov = ce depinde de ea; panoul din dreapta arată interogările folosite. „Colorează după student”: verde promovat, roșu restanță, albastru poate_inscrie
Studenți	câte un card pentru fiecare student: colegi, restanțe, înscrieri posibile, eligibilitate	„Vezi în graf” colorează graful; butonul de trace explică eligibilitatea
Baza de cunoștințe	lista predicatelor (fapte / reguli) și clauzele lor	click pe un predicat, ca `:list nume`
Ajutor	sintaxa, predicatele, semnificația trace-ului	-

Linkuri directe (utile la demo, se pot pune ca favorite în browser):

Link	Efect
`http://127.0.0.1:8000/#examples`	deschide tab-ul Exemple (la fel `#graph`, `#students`, `#kb`, `#help`)
`#q=colegi(X, Y).`	rulează direct interogarea
`#t=eligibil_licenta(emil).`	rulează interogarea cu trace
`#student=cristian`	graful colorat după situația lui Cristian
`#subject=sisteme_incorporate`	graful cu materia selectată

2.4 Testele automate

```
pytest -v
```

Fișier	Ce verifică
`tests/test_terms.py`	crearea termenilor, egalitatea, afișarea în sintaxa Prolog
`tests/test_unify.py`	unificarea: constante, variabile, functor/aritate diferită, `p(X, X)` cu `p(a, b)`, occurs check, lanțuri de variabile, substituția inițială nemodificată
`tests/test_parser.py`	fapte, reguli, comentarii, `\=`, `not`, variabile anonime, erori cu numărul liniei
`tests/test_knowledge_base.py`	indexarea, ordinea clauzelor, încărcarea din fișier; consistența KB: credite pentru fiecare materie, prerechizite doar din semestre anterioare, lanț de minim 4 niveluri
`tests/test_engine.py`	fapte, conjuncții, recursivitate, redenumire, `\=`, `not`, limita de adâncime, trace-ul, răspunsurile pe rând; toate regulile pe curriculum.kb
`tests/test_main.py`	REPL-ul: afișarea răspunsurilor, true/false, erori, `:list`, `:trace`, `:help`, `:quit`
`tests/test_web.py`	API-ul web, toate cele 40 de exemple, serverul HTTP pornit pe un port liber

3. Sintaxa limbajului

Element	Sintaxă	FOL	Exemplu
constantă	literă mică sau număr	constantă	`andrei`, `baze_de_date`, `4`
variabilă	literă mare sau _	variabilă	`S`, `Materie`, `_Ceva`
variabilă anonimă	`_`	variabilă nouă de fiecare dată	`student(S, _)`
fapt	`p(a, b).`	$p(a, b)$	`student(andrei, 1).`
regulă	`cap :- s1, s2.`	$\forall\, (s1 \wedge s2 \rightarrow cap)$	`restanta(S, M) :- picat(S, M), not(promovat(S, M)).`
conjuncție	`,`	$\wedge$	`student(S, 2), promovat(S, M)`
diferit	`X \= Y`	$X \neq Y$	`S1 \= S2`
negație	`not(scop)`	$\neg$ (negație ca eșec)	`not(picat(S, _))`
comentariu	`%` până la final de linie	-	`% anul 1`

Toate regulile sunt **clauze Horn**: concluzia are un singur predicat. Variabilele din capul regulii sunt cuantificate universal. Variabilele care apar doar în corp sunt, echivalent, cuantificate existențial în corp:

$$\forall X\, \forall Y\, (\, \exists Z\, (\, p(X, Z) \wedge q(Z, Y)\,) \rightarrow r(X, Y)\,) \;\equiv\; \forall X\, \forall Y\, \forall Z\, (\, p(X, Z) \wedge q(Z, Y) \rightarrow r(X, Y)\,)$$

Negația ca eșec și lumea închisă. `not(scop)` reușește dacă motorul **nu poate demonstra** scopul. Se presupune că baza de cunoștințe conține tot ce e adevărat (Closed World Assumption): `promovat(andrei, fizica_1)` lipsește, deci e considerat fals. De aceea `not` și `\=` stau la sfârșitul corpului regulilor, unde variabilele sunt deja legate.

4. Faptele: materii, prerechizite, studenți

4.1 Toate materiile

Fiecare materie apare ca `materie(Cod, An, Semestru)` și `credite(Cod, N)`; opționalele au în plus `optional(Cod)`. Creditele sunt valori ECTS plauzibile (30 pe semestru).

Sem	Cod în KB	Denumire	Credite	Opț.
1	analiza_matematica	Analiză matematică	6	
1	algebra_liniara	Algebră liniară și geometrie	5	
1	programarea_calculatoarelor	Programarea calculatoarelor	6	
1	proiectare_logica_1	Proiectare logică I	5	
1	fizica_1	Fizică I - Elemente generale	4	
1	introducere_ti	Introducere în tehnologia informației	2	
1	engleza_1	Engleză 1	2	
1	educatie_fizica_1	Educație fizică I	2	
2	matematici_speciale	Matematici speciale	5	
2	metode_numerice	Metode numerice	4	
2	tehnici_de_programare	Programarea calculatoarelor - Tehnici de programare	6	
2	proiectare_logica_2	Proiectare logică II	5	
2	electrotehnica	Electrotehnică	5	
2	fizica_2	Fizică II - Elemente de mecanică	3	
2	engleza_2	Engleză 2	2	
2	educatie_fizica_2	Educație fizică II	2	
3	structuri_de_date	Structuri de date	6	
3	arhitectura_sistemelor_de_calcul	Arhitectura sistemelor de calcul	5	
3	programarea_orientata_obiect	Programarea orientată pe obiecte	6	
3	electronica	Electronică	5	
3	teoria_sistemelor	Teoria sistemelor	4	
3	economie_contabilitate	Economie generală și contabilitate	2	
3	engleza_3	Engleză 3	2	
3	educatie_fizica_3	Educație fizică III	2	
4	structura_organizarea_calculatoarelor	Structura și organizarea calculatoarelor	5	
4	inteligenta_artificiala	Inteligență artificială	5	
4	grafica_pe_calculator	Elemente de grafică pe calculator	4	
4	proiectarea_aplicatiilor_oo	Proiectarea aplicațiilor orientate pe obiecte	5	
4	grafica_asistata	Grafică asistată de calculator	3	
4	masuratori_senzori_traductoare	Măsurători electronice, senzori și traductoare	4	
4	engleza_4	Engleză 4	2	

Sem	Cod în KB	Denumire	Credite	Opț.
4	practica_1	Practică 1	2	
4	educatie_fizica_4	Educație fizică IV	2	
5	circuite_integrate_digitale	Circuite integrate digitale	5	
5	baze_de_date	Baze de date	5	
5	sisteme_de_operare	Sisteme de operare	5	
5	retele_de_calculatoare	Rețele de calculatoare	5	
5	sisteme_concurente_distribuite	Sisteme concurente și distribuite	4	
5	modelare_simulare	Modelare și simulare	4	
5	proiect_1	Proiect I - Sisteme de calcul	2	
6	algoritmi_paraleli_distribuiti	Algoritmi paraleli și distribuiți	4	
6	ingineria_programarii	Ingineria programării	4	
6	proiectarea_aplicatiilor_web	Proiectarea aplicațiilor Web	4	
6	comunicatii_de_date	Comunicații de date	4	
6	proiectarea_cu_microprocesoare	Proiectarea cu microprocesoare	5	
6	limbaje_formale_automate	Limbaje formale și automate	4	
6	proiect_2	Proiect II - Sisteme informatice	2	
6	practica_2	Practică 2	3	
7	managementul_proiectelor	Managementul proiectelor	4	
7	comert_electronic	Comerț electronic	4	
7	securitatea_datelor	Securitatea datelor	5	
7	proiectarea_bazelor_de_date	Proiectarea bazelor de date	5	
7	proiectarea_translatoarelor	Proiectarea translatoarelor	5	
7	invatare_automata	Învățare automată	5	
7	proiect_3	Proiect III - Tehnologia informației	2	
8	administrarea_retelelor	Administrarea rețelelor de calculatoare	4	
8	circuite_vlsi	Circuite VLSI	4	
8	verificarea_testarea_sistemelor	Verificarea și testarea sistemelor de calcul	4	
8	proiect_4	Proiect IV - Microcalculatoare	2	
8	practica_licenta	Practică pentru elaborare proiect licență	4	
8	calcul_mobil	Calcul mobil	4	da
8	regasirea_informatiei	Regăsirea informației	4	da
8	sisteme_grafice	Sisteme grafice	4	da
8	sisteme_multimedia	Sisteme multimedia	4	da
8	interactiunea_om_calculator	Interacțiunea om-calculator	4	da
8	sisteme_incorporate	Sisteme încorporate	4	da
8	procesarea_semnalelor	Procesarea numerică a semnalelor	4	da
8	retele_de_senzori	Rețele de senzori	4	da
8	sisteme_timp_real	Sisteme de calcul în timp real	4	da

4.2 Toate legăturile de prerechizit

`prerechizit(P, M)` înseamnă că P trebuie promovată înainte de M. Orice prerechizit e într-un semestru **anterior** materiei care îl cere (verificat automat de teste). Tabelul de mai jos are fiecare materie care are prerechizite, cu toate prerechizitele ei directe.

Sem	Materia	Prerechizite directe
2	Programarea calculatoarelor - Tehnici de programare	Programarea calculatoarelor (sem 1)
2	Proiectare logică II	Proiectare logică I (sem 1)
2	Electrotehnică	Fizică I - Elemente generale (sem 1)
2	Matematici speciale	Analiză matematică (sem 1)
2	Metode numerice	Analiză matematică (sem 1)
2	Engleză 2	Engleză 1 (sem 1)
2	Educație fizică II	Educație fizică I (sem 1)
3	Structuri de date	Programarea calculatoarelor - Tehnici de programare (sem 2)
3	Programarea orientată pe obiecte	Programarea calculatoarelor - Tehnici de programare (sem 2)
3	Arhitectura sistemelor de calcul	Proiectare logică II (sem 2)
3	Electronică	Electrotehnică (sem 2)
3	Teoria sistemelor	Matematici speciale (sem 2)
3	Engleză 3	Engleză 2 (sem 2)
3	Educație fizică III	Educație fizică II (sem 2)
4	Proiectarea aplicațiilor orientate pe obiecte	Programarea orientată pe obiecte (sem 3)
4	Inteligență artificială	Structuri de date (sem 3)
4	Structura și organizarea calculatoarelor	Arhitectura sistemelor de calcul (sem 3)
4	Măsurători electronice, senzori și traductoare	Electronică (sem 3)
4	Elemente de grafică pe calculator	Algebră liniară și geometrie (sem 1); Programarea calculatoarelor - Tehnici de programare (sem 2)
4	Engleză 4	Engleză 3 (sem 3)
4	Educație fizică IV	Educație fizică III (sem 3)
5	Baze de date	Structuri de date (sem 3)
5	Sisteme concurente și distribuite	Structuri de date (sem 3); Programarea orientată pe obiecte (sem 3)
5	Sisteme de operare	Structura și organizarea calculatoarelor (sem 4)
5	Rețele de calculatoare	Structura și organizarea calculatoarelor (sem 4)
5	Proiect I - Sisteme de calcul	Arhitectura sistemelor de calcul (sem 3)
5	Circuite integrate digitale	Electronică (sem 3); Proiectare logică II (sem 2)
5	Modelare și simulare	Teoria sistemelor (sem 3)
6	Ingineria programării	Proiectarea aplicațiilor orientate pe obiecte (sem 4)
6	Proiectarea aplicațiilor Web	Proiectarea aplicațiilor orientate pe obiecte (sem 4)
6	Proiect II - Sisteme informatice	Baze de date (sem 5)
6	Algoritmi paraleli și distribuiți	Sisteme concurente și distribuite (sem 5)
6	Limbaje formale și automate	Structuri de date (sem 3)

Sem	Materia	Prerechizite directe
6	Proiectarea cu microprocesoare	Structura și organizarea calculatoarelor (sem 4)
6	Comunicații de date	Rețele de calculatoare (sem 5)
6	Practică 2	Practică 1 (sem 4)
7	Comerț electronic	Proiectarea aplicațiilor Web (sem 6)
7	Proiect III - Tehnologia informației	Proiectarea aplicațiilor Web (sem 6)
7	Managementul proiectelor	Ingineria programării (sem 6)
7	Proiectarea bazelor de date	Baze de date (sem 5)
7	Proiectarea translatoarelor	Limbaje formale și automate (sem 6)
7	Învățare automată	Inteligență artificială (sem 4); Metode numerice (sem 2)
7	Securitatea datelor	Rețele de calculatoare (sem 5)
8	Calcul mobil	Proiectarea aplicațiilor Web (sem 6)
8	Interacțiunea om-calculator	Proiectarea aplicațiilor Web (sem 6)
8	Regăsirea informației	Învățare automată (sem 7)
8	Sisteme încorporate	Proiectarea cu microprocesoare (sem 6)
8	Proiect IV - Microcalculatoare	Proiectarea cu microprocesoare (sem 6)
8	Sisteme de calcul în timp real	Proiectarea cu microprocesoare (sem 6); Sisteme de operare (sem 5)
8	Circuite VLSI	Circuite integrate digitale (sem 5)
8	Verificarea și testarea sistemelor de calcul	Circuite integrate digitale (sem 5)
8	Procesarea numerică a semnalelor	Teoria sistemelor (sem 3)
8	Administrarea rețelelor de calculatoare	Rețele de calculatoare (sem 5)
8	Rețele de senzori	Rețele de calculatoare (sem 5); Măsurători electronice, senzori și traductoare (sem 4)
8	Sisteme grafice	Elemente de grafică pe calculator (sem 4)
8	Sisteme multimedia	Elemente de grafică pe calculator (sem 4)
8	Practică pentru elaborare proiect licență	Practică 2 (sem 6)

Lanțurile principale (citite de la stânga la dreapta):

- **Hardware:** Proiectare logică I → Proiectare logică II → Arhitectura sistemelor de calcul → Structura și organizarea calculatoarelor → Proiectarea cu microprocesoare → Sisteme încorporate (cel mai lung lanț, 6 niveluri)
- **Programare:** Programarea calculatoarelor → Tehnici de programare → POO → Proiectarea aplicațiilor OO → Proiectarea aplicațiilor Web → Comerț electronic
- **Date și IA:** Tehnici de programare → Structuri de date → Inteligență artificială → Învățare automată → Regăsirea informației
- **Electronică:** Fizică I → Electrotehnică → Electronică → Circuite integrate digitale → Circuite VLSI
- **Rețele:** Structura și organizarea calculatoarelor → Rețele de calculatoare → Securitatea datelor / Comunicații de date / Administrarea rețelelor
- **Matematică și sisteme:** Analiză matematică → Matematici speciale → Teoria sistemelor → Modelare și simulare / Procesarea numerică a semnalelor
- **Limbi, sport, practică:** Engleză 1 → 2 → 3 → 4; Educație fizică I → II → III → IV; Practică 1 → Practică 2 → Practică pentru licență

4.3 Studenții

Student	An	Promovate	Picate	Restanțe	Eligibil licență
andrei	1	7	fizica_1	fizica_1	nu
bianca	2	16	-	-	nu
cristian	2	15	proiectare_logica_2	proiectare_logica_2	nu
diana	4	56	-	-	da
emil	4	55	securitatea_datelor	securitatea_datelor	nu

andrei (anul 1)

anul 1, a terminat doar semestrul 1 și a picat Fizica I; îi lipsesc multe, deci e bun pentru exemplele cu ii_lipseste și pentru testul „toate prerechizitele”.

Promovat (7): analiza_matematica, algebra_liniara, programarea_calculatoarelor, proiectare_logica_1, introducere_ti, engleza_1, educatie_fizica_1

Se poate înscrie la (11): fizica_1, matematici_speciale, metode_numerice, tehnici_de_programare, proiectare_logica_2, fizica_2, engleza_2, educatie_fizica_2, economie_contabilitate, grafica_asistata, practica_1

bianca (anul 2)

anul 2, tot anul 1 promovat: studentul „fără probleme”.

Promovat (16): analiza_matematica, algebra_liniara, programarea_calculatoarelor, proiectare_logica_1, fizica_1, introducere_ti, engleza_1, educatie_fizica_1, matematici_speciale, metode_numerice, tehnici_de_programare, proiectare_logica_2, electrotehnica, fizica_2, engleza_2, educatie_fizica_2

Se poate înscrie la (11): structuri_de_date, arhitectura_sistemelor_de_calcul, programarea_orientata_obiect, electronica, teoria_sistemelor, economie_contabilitate, engleza_3, educatie_fizica_3, grafica_pe_calculator, grafica_asistata, practica_1

cristian (anul 2)

anul 2, a picat Proiectare logică II; restanța îi blochează Arhitectura și, prin lanț, mai multe materii din anii 3–4.

Promovat (15): analiza_matematica, algebra_liniara, programarea_calculatoarelor, proiectare_logica_1, fizica_1, introducere_ti, engleza_1, educatie_fizica_1, matematici_speciale, metode_numerice, tehnici_de_programare, electrotehnica, fizica_2, engleza_2, educatie_fizica_2

Se poate înscrie la (11): proiectare_logica_2, structuri_de_date, programarea_orientata_obiect, electronica, teoria_sistemelor, economie_contabilitate, engleza_3, educatie_fizica_3, grafica_pe_calculator, grafica_asistata, practica_1

diana (anul 4)

anul 4, fără restanțe, a făcut practica de licență: singura eligibilă pentru licență.

Promovat (56): analiza_matematica, algebra_liniara, programarea_calculatoarelor, proiectare_logica_1, fizica_1, introducere_ti, engleza_1, educatie_fizica_1, matematici_speciale, metode_numerice, tehnici_de_programare, proiectare_logica_2, electrotehnica, fizica_2, engleza_2, educatie_fizica_2, structuri_de_date, arhitectura_sistemelor_de_calcul, programarea_orientata_obiect, electronica, teoria_sistemelor, economie_contabilitate, engleza_3, educatie_fizica_3, structura_organizarea_calculatoarelor, inteligenta_artificiala, grafica_pe_calculator, proiectarea_aplicatiilor_oo, grafica_asistata, masuratori_senzori_traductoare, engleza_4, practica_1, educatie_fizica_4, circuite_integrate_digitale, baze_de_date, sisteme_de_operare, retele_de_calculatoare, sisteme_concurente_distribuite, modelare_simulare, proiect_1, algoritmi_paraleli_distribuiti, ingineria_programarii, proiectarea_aplicatiilor_web, comunicatii_de_date, proiectarea_cu_microprocesoare, limbaje_formale_automate, proiect_2, practica_2, managementul_proiectelor, comert_electronic, securitatea_datelor, proiectarea_bazelor_de_date, proiectarea_translatoarelor, invatare_automata, proiect_3, practica_licenta

Se poate înscrie la (13): administrarea_retelelor, circuite_vlsi, verificarea_testarea_sistemelor, proiect_4, calcul_mobil, regasirea_informatiei, sisteme_grafice, sisteme_multimedia, interactiunea_om_calculator, sisteme_incorporate, procesarea_semnalelor, retele_de_senzori, sisteme_timp_real

emil (anul 4)

anul 4, identic cu Diana, dar a picat Securitatea datelor: exemplul pentru not(restanta(...)).

Promovat (55): analiza_matematica, algebra_liniara, programarea_calculatoarelor, proiectare_logica_1, fizica_1, introducere_ti, engleza_1, educatie_fizica_1, matematici_speciale, metode_numerice, tehnici_de_programare, proiectare_logica_2, electrotehnica, fizica_2, engleza_2, educatie_fizica_2, structuri_de_date, arhitectura_sistemelor_de_calcul, programarea_orientata_obiect, electronica, teoria_sistemelor, economie_contabilitate, engleza_3, educatie_fizica_3, structura_organizarea_calculatoarelor, inteligenta_artificiala, grafica_pe_calculator, proiectarea_aplicatiilor_oo, grafica_asistata, masuratori_senzori_traductoare, engleza_4, practica_1, educatie_fizica_4, circuite_integrate_digitale, baze_de_date, sisteme_de_operare, retele_de_calculatoare, sisteme_concurente_distribuite, modelare_simulare, proiect_1, algoritmi_paraleli_distribuiti, ingineria_programarii, proiectarea_aplicatiilor_web, comunicatii_de_date, proiectarea_cu_microprocesoare, limbaje_formale_automate, proiect_2, practica_2, managementul_proiectelor, comert_electronic, proiectarea_bazelor_de_date, proiectarea_translatoarelor, invatare_automata, proiect_3, practica_licenta

Se poate înscrie la (14): securitatea_datelor, administrarea_retelelor, circuite_vlsi, verificarea_testarea_sistemelor, proiect_4, calcul_mobil, regasirea_informatiei, sisteme_grafice, sisteme_multimedia, interactiunea_om_calculator, sisteme_incorporate, procesarea_semnalelor, retele_de_senzori, sisteme_timp_real

5. Regulile

Pentru fiecare regulă: semnificația, formula în FOL (cu cuantificatori), forma din `kb/curriculum.kb` și cum o folosește motorul.

5.1 necesita(M, P): prerechizitele directe și indirecte (recursiv)

P este necesară pentru M dacă e prerechizit direct al lui M, sau dacă e necesară pentru un prerechizit direct al lui M. Este închiderea tranzitivă a relației prerechizit.

FOL:

$$\forall M\ \forall P\ (\ \text{prerechizit}(P, M) \rightarrow \text{necesita}(M, P)\ )$$

$$\forall M\ \forall P\ \forall Q\ (\ \text{prerechizit}(Q, M) \land \text{necesita}(Q, P) \rightarrow \text{necesita}(M, P)\ )$$

În baza de cunoștințe:

```
necesita(M, P) :- prerechizit(P, M).
necesita(M, P) :- prerechizit(Q, M), necesita(Q, P).
```

Cazul de bază e primul, iar recursia e **la dreapta**: `prerechizit(Q, M)` leagă întâi `Q`, apoi se face apelul recursiv. Varianta `necesita(M, P) :- necesita(Q, P), prerechizit(Q, M).` (recursie la stânga) ar intra în buclă infinită la căutarea în adâncime.

5.2 poate_inscrie(S, M): are TOATE prerechizitele directe

Studentul S se poate înscrie la M dacă nu a promovat-o deja și a promovat toate prerechizitele directe ale lui M.

FOL:

$$\forall S\ \forall M\ (\ \exists A\ \text{student}(S, A) \land \exists An\ \exists Sem\ \text{materie}(M, An, Sem) \land \neg\text{promovat}(S, M) \land \forall P\ (\ \text{prerechizit}(P, M) \rightarrow \text{promovat}(S, P)\ ) \rightarrow \text{poate_inscrie}(S, M)\ )$$

Rescriere: $\forall P\ (\ \text{prerechizit}(P, M) \rightarrow \text{promovat}(S, P)\ ) \equiv \neg\exists P\ (\ \text{prerechizit}(P, M) \land \neg\text{promovat}(S, P)\ )$

$$\forall S\ \forall M\ \forall P\ (\ \text{prerechizit}(P, M) \land \neg\text{promovat}(S, P) \rightarrow \text{prerechizit_nepromovat}(S, M)\ )$$

În baza de cunoștințe:

```
poate_inscrie(S, M) :- student(S, _), materie(M, _, _), not(promovat(S, M)),
not(prerechizit_nepromovat(S, M)).
prerechizit_nepromovat(S, M) :- prerechizit(P, M), not(promovat(S, P)).
```

O clauză Horn nu poate conține $\forall$ în corp. Folosim echivalența „toate sunt promovate” = „nu există unul nepromovat” ($\forall = \neg\exists\neg$) și un predicat ajutător pentru partea cu $\exists$. `student(S, _)` și `materie(M, _, _)` stau primele ca să lege S și M înainte de negații.

5.3 ii_lipseste(S, M, P): ce îi lipsește din tot lanțul

Pentru materia M, studentului S îi lipsește P dacă P e în lanțul de prerechizite al lui M (direct sau indirect) și S nu a promovat-o.

FOL:

$$\forall S\ \forall M\ \forall P\ (\ \exists A\ \text{student}(S, A) \land \text{necesita}(M, P) \land \neg\text{promovat}(S, P) \rightarrow \text{ii_lipseste}(S, M, P)\ )$$

În baza de cunoștințe:

```
ii_lipseste(S, M, P) :- student(S, _), necesita(M, P), not(promovat(S, P)).
```

Combină regula recursivă `necesita` cu negația: un lanț de reguli.

5.4 colegi(S1, S2): același an, persoane diferite

Doi studenți sunt colegi dacă sunt în același an de studiu și nu sunt aceeași persoană.

FOL:

$$\forall S1\ \forall S2\ \forall A\ (\ \text{student}(S1, A) \land \text{student}(S2, A) \land S1 \neq S2 \rightarrow \text{colegi}(S1, S2)\ )$$

În baza de cunoștințe:

```
colegi(S1, S2) :- student(S1, An), student(S2, An), S1 \= S2.
```

Variabila comună `An` obligă cei doi studenți să fie din același an. `S1 \= S2` elimină perechea unui student cu el însuși.

5.5 restanta(S, M): a picat și nu a promovat încă

S are restanță la M dacă a picat examenul și nu l-a promovat ulterior.

FOL:

$$\forall S\ \forall M\ (\ \text{picat}(S, M) \land \neg\text{promovat}(S, M) \rightarrow \text{restanta}(S, M)\ )$$

În baza de cunoștințe:

```
restanta(S, M) :- picat(S, M), not(promovat(S, M)).
```

Dacă s-ar adăuga faptul `promovat(emil, securitatea_datelor).`, restanța lui Emil ar dispărea automat, fără să modificăm faptul `picat`.

5.6 eligibil_licenta(S): poate susține licența

S e eligibil dacă e în anul 4, a făcut practica de licență și nu are nicio restanță.

FOL:

$$\forall S\ (\ \text{student}(S, 4) \land \text{promovat}(S, \text{practica_licenta}) \land \neg\exists M\ \text{restanta}(S, M) \rightarrow \text{eligibil_licenta}(S)\ )$$

În baza de cunoștințe:

```
eligibil_licenta(S) :- student(S, 4), promovat(S, practica_licenta), not(restanta(S, _)).
```

`not(restanta(S, _))` cu variabila anonimă înseamnă exact ¬ ∃ M restanta(S, M): motorul caută orice restanță și, dacă găsește una, negația eșuează. Sunt două negații imbricate, pentru că `restanta` conține la rândul ei un `not`.

5.7 Graful de dependențe între reguli

```
eligibil_licenta       ⟶ student, promovat, not(restanta)
restanta               ⟶ picat, not(promovat)
colegi                 ⟶ student, \=
ii_lipseste            ⟶ student, necesita, not(promovat)
necesita               ⟶ prerechizit, necesita (recursiv)
poate_inscrie          ⟶ student, materie, not(promovat), not(prerechizit_nepromovat)
prerechizit_nepromovat ⟶ prerechizit, not(promovat)
```

6. Exemple de interogări cu rezultatele reale

Toate cele 40 de exemple au fost rulate prin motor la generarea acestui document; rezultatele sunt exact cele afișate de program. Aceleași exemple, cu aceeași numerotare, se pot rula cu un click din tab-ul **Exemple** al interfeței web, sau se pot scrie în terminal după `python main.py`.

Categorie	Exemple	Ce demonstrează
Fapte simple	#1–#6	unificarea cu fapte, conjuncții simple
Recursivitate: necesita	#7–#11	regula recursivă, folosită în ambele direcții
Toate prerechizitele: poate_inscrie	#12–#17	„pentru toate” prin dublă negație
Lanțuri de reguli: ii_lipseste	#18–#21	lanțuri de reguli: recursivitate + negație
Diferit: \=	#22–#25	operatorul \= și backtracking-ul
Negație ca eșec: not	#26–#33	negația ca eșec, lumea închisă, afirmații universale
Interogări compuse	#34–#40	interogări cu 3–5 variabile și mai multe predicate

Fapte simple

#1 Toți studenții și anul lor

```
?- student(S, An).
S = andrei, An = 1
S = bianca, An = 2
S = cristian, An = 2
S = diana, An = 4
S = emil, An = 4
```

Cea mai simplă interogare: două variabile, unificate cu fiecare fapt student/2, în ordinea din fișier. (5 răspunsuri)

#2 Materiile din anul 1, semestrul 1

```
?- materie(M, 1, 1).
M = analiza_matematica
M = algebra_liniara
M = programarea_calculatoarelor
M = proiectare_logica_1
M = fizica_1
M = introducere_ti
M = engleza_1
M = educatie_fizica_1
```

Constantele 1 și 1 filtrează faptele, iar variabila M primește doar materiile care se potrivesc. (8 răspunsuri)

#3 Câte credite are Baze de date

```
?- credite(baze_de_date, C).
C = 5
```

Căutare directă a unei valori. (1 răspuns)

#4 Ce a promovat Andrei

```
?- promovat(andrei, M).
M = analiza_matematica
M = algebra_liniara
M = programarea_calculatoarelor
M = proiectare_logica_1
M = introducere_ti
M = engleza_1
M = educatie_fizica_1
```

Lipsește fizica_1: Andrei a picat-o, iar ipoteza lumii închise spune că ce nu e scris nu e adevărat. (7 răspunsuri)

#5 Toate examenele picate

```
?- picat(S, M).
S = andrei, M = fizica_1
S = cristian, M = proiectare_logica_2
S = emil, M = securitatea_datelor
```

Trei fapte picat/2, unul pentru fiecare student cu probleme. (3 răspunsuri)

#6 Opționalele din semestrul 8

```
?- materie(M, An, 8), optional(M).
M = calcul_mobil, An = 4
M = regasirea_informatiei, An = 4
M = sisteme_grafice, An = 4
M = sisteme_multimedia, An = 4
M = interactiunea_om_calculator, An = 4
M = sisteme_incorporate, An = 4
M = procesarea_semnalelor, An = 4
M = retele_de_senzori, An = 4
M = sisteme_timp_real, An = 4
```

Conjuncție de două fapte legate prin variabila comună M. (9 răspunsuri)

Recursivitate: necesita

#7 Tot lanțul pentru Sisteme încorporate

```
?- necesita(sisteme_incorporate, P).
P = proiectarea_cu_microprocesoare
P = structura_organizarea_calculatoarelor
P = arhitectura_sistemelor_de_calcul
P = proiectare_logica_2
P = proiectare_logica_1
```

În KB e scris un singur prerechizit direct (proiectarea_cu_microprocesoare); celelalte 4 materii sunt deduse aplicând regula recursivă de mai multe ori. (5 răspunsuri)

#8 Tot lanțul pentru Baze de date

```
?- necesita(baze_de_date, P).
P = structuri_de_date
P = tehnici_de_programare
P = programarea_calculatoarelor
```

Structuri de date vine din KB, iar Tehnici de programare și Programarea calculatoarelor sunt deduse prin recursivitate. (3 răspunsuri)

#9 Ce depinde de Proiectare logică I

```
?- necesita(M, proiectare_logica_1).
M = proiectare_logica_2
M = arhitectura_sistemelor_de_calcul
M = structura_organizarea_calculatoarelor
M = proiectarea_cu_microprocesoare
M = sisteme_de_operare
M = retele_de_calculatoare
M = proiect_1
M = sisteme_incorporate
M = proiect_4
M = sisteme_timp_real
M = sisteme_timp_real
M = circuite_integrate_digitale
M = circuite_vlsi
M = verificarea_testarea_sistemelor
M = comunicatii_de_date
M = securitatea_datelor
M = administrarea_retelelor
M = retele_de_senzori
```

Aceeași regulă, folosită invers. sisteme_timp_real apare de două ori pentru că are două demonstrații diferite: prin microprocesoare și prin sisteme de operare. (18 răspunsuri)

#10 Regăsirea informației depinde de Programarea calculatoarelor?

```
?- necesita(regasirea_informatiei, programarea_calculatoarelor).
true.
```

Întrebare da/nu. Lanțul are 6 niveluri: PC, TP, SD, IA, Învățare automată, Regăsirea informației.

#11 Planul are cicluri?

```
?- necesita(X, X).
false.
```

false înseamnă că nicio materie nu depinde de ea însăși: graful de prerechizite e aciclic.

Toate prerechizitele: poate_inscrie

#12 La ce se poate înscrie Cristian

```
?- poate_inscrie(cristian, M).
M = proiectare_logica_2
M = structuri_de_date
M = programarea_orientata_obiect
M = electronica
M = teoria_sistemelor
M = economie_contabilitate
M = engleza_3
M = educatie_fizica_3
M = grafica_pe_calculator
M = grafica_asistata
M = practica_1
```

Lipsește arhitectura_sistemelor_de_calcul: îi cere proiectare_logica_2, pe care Cristian a picat-o. Apare în schimb proiectare_logica_2, pe care o poate reface. (11 răspunsuri)

#13 La ce se poate înscrie Andrei

```
?- poate_inscrie(andrei, M).
M = fizica_1
M = matematici_speciale
M = metode_numerice
M = tehnici_de_programare
M = proiectare_logica_2
M = fizica_2
M = engleza_2
M = educatie_fizica_2
M = economie_contabilitate
M = grafica_asistata
M = practica_1
```

Apare fizica_1, pe care a picat-o și o poate reface. Apar și câteva materii din anul 2 fără prerechizite, pentru că regula nu verifică anul. (11 răspunsuri)

#14 Ce poate alege Diana în ultimul semestru

```
?- poate_inscrie(diana, M).
M = administrarea_retelelor
M = circuite_vlsi
M = verificarea_testarea_sistemelor
M = proiect_4
M = calcul_mobil
M = regasirea_informatiei
M = sisteme_grafice
M = sisteme_multimedia
M = interactiunea_om_calculator
M = sisteme_incorporate
M = procesarea_semnalelor
M = retele_de_senzori
M = sisteme_timp_real
```

Toate materiile din semestrul 8, inclusiv cele 9 opționale: are tot ce trebuie. (13 răspunsuri)

#15 Cine se poate înscrie la Structuri de date

```
?- poate_inscrie(S, structuri_de_date).
S = bianca
S = cristian
```

Aceeași regulă, cu studentul necunoscut. Diana și Emil n-o pot face pentru că au promovat-o deja. (2 răspunsuri)

#16 Bianca la Grafică pe calculator

```
?- poate_inscrie(bianca, grafica_pe_calculator).
true.
```

grafica_pe_calculator are două prerechizite (algebra_liniara și tehnici_de_programare). Bianca le are pe amândouă.

#17 Andrei la Grafică pe calculator

```
?- poate_inscrie(andrei, grafica_pe_calculator).
false.
```

Andrei are doar algebra_liniara. Regula cere TOATE prerechizitele ($\forall$ P = $\neg$ $\exists$ P nepromovat), deci răspunsul e false. O regulă naivă cu un singur prerechizit ar fi zis true.

Lanțuri de reguli: ii_lipseste

#18 Ce îi lipsește lui Cristian pentru Circuite VLSI

```
?- ii_lipseste(cristian, circuite_vlsi, P).
P = circuite_integrate_digitale
P = electronica
P = proiectare_logica_2
```

Lanțul complet are 6 materii. not(promovat(...)) le filtrează pe cele 3 pe care Cristian le are deja. (3 răspunsuri)

#19 Ce îi lipsește lui Andrei pentru Învățare automată, cu credite

```
?- ii_lipseste(andrei, invatare_automata, P), credite(P, C).
P = inteligenta_artificiala, C = 5
P = metode_numerice, C = 4
P = structuri_de_date, C = 6
P = tehnici_de_programare, C = 6
```

Regulă recursivă + negație + legătură cu predicatul credite, într-o singură interogare. (4 răspunsuri)

#20 Cui îi lipsește PL II pentru Sisteme încorporate

```
?- ii_lipseste(S, sisteme_incorporate, proiectare_logica_2).
S = andrei
S = cristian
```

Aceeași regulă, întrebată pe altă poziție: studentul e acum necunoscut. (2 răspunsuri)

#21 Ce opționale îi blochează lui Cristian restanța la PL II

```
?- optional(M), ii_lipseste(cristian, M, proiectare_logica_2).
M = sisteme_incorporate
M = retele_de_senzori
M = sisteme_timp_real
M = sisteme_timp_real
```

O singură materie picată în anul 1 blochează, prin lanțul de prerechizite, trei opționale din anul 4. (4 răspunsuri)

Diferit: \=

#22 Toate perechile de colegi

```
?- colegi(X, Y).
X = bianca, Y = cristian
X = cristian, Y = bianca
X = diana, Y = emil
X = emil, Y = diana
```

Fără X \= Y, fiecare student ar fi coleg cu el însuși. Andrei nu apare: e singurul în anul 1. (4 răspunsuri)

#23 Colegii Biancăi

```
?- colegi(bianca, X).
X = cristian
```

Cu :trace se vede backtracking-ul: întâi X = bianca, apoi bianca \= bianca eșuează și motorul trece la cristian. (1 răspuns)

#24 Colegii lui Andrei

```
?- colegi(andrei, X).
false.
```

false: nimeni altcineva nu e în anul 1.

#25 Materii deblocate de Tehnici de programare, două câte două

```
?- prerechizit(tehnici_de_programare, M), prerechizit(tehnici_de_programare, N), M \= N.
M = structuri_de_date, N = programarea_orientata_obiect
M = structuri_de_date, N = grafica_pe_calculator
M = programarea_orientata_obiect, N = structuri_de_date
M = programarea_orientata_obiect, N = grafica_pe_calculator
M = grafica_pe_calculator, N = structuri_de_date
M = grafica_pe_calculator, N = programarea_orientata_obiect
```

\= elimină perechile de materii identice. Fiecare pereche apare în ambele ordini. (6 răspunsuri)

Negație ca eșec: not

#26 Cine poate susține licența

```
?- eligibil_licenta(S).
S = diana
```

Doar Diana. Emil are o restanță, deci not(restanta(emil, _)) eșuează. (1 răspuns)

#27 Emil poate susține licența?

```
?- eligibil_licenta(emil).
false.
```

Rulează cu trace: se văd două negații imbricate, not(restanta(...)) care conține not(promovat(...)).

#28 Toate restanțele

```
?- restanta(S, M).
S = andrei, M = fizica_1
S = cristian, M = proiectare_logica_2
S = emil, M = securitatea_datelor
```

restanta = picat ȘI nu a promovat ulterior. (3 răspunsuri)

#29 Studenții care n-au picat niciodată nimic

```
?- student(S, An), not(picat(S, _)).
S = bianca, An = 2
S = diana, An = 4
```

not(picat(S, _)) înseamnă ¬ ∃ M picat(S, M). S e deja legat când se ajunge la not. (2 răspunsuri)

#30 Cine are deja tot ce trebuie pentru Baze de date

```
?- student(S, _), not(ii_lipseste(S, baze_de_date, _)).
S = diana
S = emil
```

Negația unei reguli recursive: „nu există nicio materie din lanț care să-i lipsească”. (2 răspunsuri)

#31 Materii din anul 1 care nu deblochează nimic

```
?- materie(M, 1, _), not(prerechizit(M, _)).
M = introducere_ti
M = fizica_2
```

Doar două materii din anul 1 nu sunt prerechizit pentru nicio altă materie. (2 răspunsuri)

#32 Orice materie din anul 3 are prerechizit?

```
?- materie(M, 3, _), not(prerechizit(_, M)).
false.
```

false demonstrează o afirmație universală: ∀ M (materie(M, 3, _) → ∃ P prerechizit(P, M)).

#33 Ce student din anul 4 nu are Securitatea datelor

```
?- student(S, 4), not(promovat(S, securitatea_datelor)).
S = emil
```

Ipoteza lumii închise: faptul promovat(emil, securitatea_datelor) lipsește, deci e considerat fals. (1 răspuns)

Interogări compuse

#34 Restanțe cu anul și semestrul materiei

```
?- restanta(S, M), materie(M, An, Sem).
S = andrei, M = fizica_1, An = 1, Sem = 1
S = cristian, M = proiectare_logica_2, An = 1, Sem = 2
S = emil, M = securitatea_datelor, An = 4, Sem = 7
```

O regulă (restanta) legată cu un fapt (materie), cu 4 variabile. (3 răspunsuri)

#35 Studenții din anul 2 și materiile de 6 credite la care se pot înscrie

```
?- student(S, 2), poate_inscrie(S, M), credite(M, 6).
S = bianca, M = structuri_de_date
S = bianca, M = programarea_orientata_obiect
S = cristian, M = structuri_de_date
S = cristian, M = programarea_orientata_obiect
```

Trei scopuri legate prin variabilele comune S și M. (4 răspunsuri)

#36 Studenți cu restanță care au și colegi

```
?- colegi(S, X), restanta(S, M).
S = cristian, X = bianca, M = proiectare_logica_2
S = emil, X = diana, M = securitatea_datelor
```

Două reguli diferite combinate: una cu \=, cealaltă cu not. (2 răspunsuri)

#37 Lanțul de 4 prerechizite către Sisteme încorporate

```
?- prerechizit(P, M), prerechizit(M, N), prerechizit(N, K), prerechizit(K, sisteme_incorporate).
P = proiectare_logica_2, M = arhitectura_sistemelor_de_calcul, N =
structura_organizarea_calculatoarelor, K = proiectarea_cu_microprocesoare
```

Lanțul scris explicit, fără regula recursivă. Motorul face backtracking prin toate faptele prerechizit până găsește singura combinație care se potrivește. (1 răspuns)

#38 Opționalele care depind de materii din primul semestru

```
?- necesita(M, P), materie(P, 1, 1), optional(M).
M = calcul_mobil, P = programarea_calculatoarelor
M = interactiunea_om_calculator, P = programarea_calculatoarelor
M = regasirea_informatiei, P = programarea_calculatoarelor
M = regasirea_informatiei, P = analiza_matematica
M = sisteme_incorporate, P = proiectare_logica_1
M = sisteme_timp_real, P = proiectare_logica_1
M = sisteme_timp_real, P = proiectare_logica_1
M = procesarea_semnalelor, P = analiza_matematica
M = retele_de_senzori, P = proiectare_logica_1
M = retele_de_senzori, P = fizica_1
M = sisteme_grafice, P = algebra_liniara
M = sisteme_grafice, P = programarea_calculatoarelor
M = sisteme_multimedia, P = algebra_liniara
M = sisteme_multimedia, P = programarea_calculatoarelor
```

Rezultatul regulii recursive e filtrat cu două fapte. Unele perechi apar de mai multe ori, câte o dată pentru fiecare drum din graf. (14 răspunsuri)

#39 Prerechizite din același an cu materia

```
?- prerechizit(P, M), materie(P, An, _), materie(M, An, _).
P = programarea_calculatoarelor, M = tehnici_de_programare, An = 1
P = programarea_orientata_obiect, M = proiectarea_aplicatiilor_oo, An = 2
P = baze_de_date, M = proiect_2, An = 3
P = sisteme_concurente_distribuite, M = algoritmi_paraleli_distribuiti, An = 3
P = structuri_de_date, M = inteligenta_artificiala, An = 2
P = invatare_automata, M = regasirea_informatiei, An = 4
P = proiectare_logica_1, M = proiectare_logica_2, An = 1
P = arhitectura_sistemelor_de_calcul, M = structura_organizarea_calculatoarelor, An = 2
P = fizica_1, M = electrotehnica, An = 1
P = electronica, M = masuratori_senzori_traductoare, An = 2
P = analiza_matematica, M = matematici_speciale, An = 1
P = analiza_matematica, M = metode_numerice, An = 1
P = retele_de_calculatoare, M = comunicatii_de_date, An = 3
P = engleza_1, M = engleza_2, An = 1
P = engleza_3, M = engleza_4, An = 2
P = educatie_fizica_1, M = educatie_fizica_2, An = 1
P = educatie_fizica_3, M = educatie_fizica_4, An = 2
```

Variabila An apare în ambele fapte materie, deci obligă cele două materii să fie din același an (prerechizitul e în semestrul 1, materia în semestrul 2). (17 răspunsuri)

#40 Înscrieri posibile în afara anului de studiu

```
?- student(S, An), poate_inscrie(S, M), materie(M, An2, _), An \= An2.
S = andrei, An = 1, M = economie_contabilitate, An2 = 2
S = andrei, An = 1, M = grafica_asistata, An2 = 2
S = andrei, An = 1, M = practica_1, An2 = 2
S = cristian, An = 2, M = proiectare_logica_2, An2 = 1
```

Arată limitarea regulii poate_inscrie: Andrei vede materii din anul 2 fără prerechizite, iar Cristian reface o materie din anul 1. (4 răspunsuri)

7. Trace-uri explicate pas cu pas

Cu `:trace` (în terminal) sau bifa **Trace** (în web), motorul afișează fiecare pas: **Call** = încearcă să demonstreze scopul; **Exit** = a reușit (scopul e afișat cu variabilele legate); **Fail** = nu există nicio soluție, deci se face backtracking. Indentarea este adâncimea în arborele de demonstrație. Variabilele cu sufix (`An_1`, `P_3`) sunt copiile create prin redenumire la fiecare folosire a unei reguli.

7.1 colegi(bianca, X): backtracking

```
Call: colegi(bianca, X)
  Call: student(bianca, An_1)
  Exit: student(bianca, 2)
  Call: student(S2_1, 2)
  Exit: student(bianca, 2)
  Call: bianca \= bianca
  Fail: bianca \= bianca
  Exit: student(cristian, 2)
  Call: bianca \= cristian
  Exit: bianca \= cristian
Exit: colegi(bianca, cristian)
```

1. Scopul se unifică cu capul regulii `colegi(S1_1, S2_1)`, deci S1_1 = bianca și S2_1 = X. Corpul regulii devine lista de scopuri.
2. `student(bianca, An_1)` găsește faptul `student(bianca, 2)`, deci An_1 = 2.
3. `student(S2_1, 2)`: primul fapt potrivit e tot `student(bianca, 2)`.
4. `bianca \= bianca` eșuează (**Fail**), deci motorul se întoarce (backtracking) la ultimul punct cu alternative.
5. Următorul fapt potrivit este `student(cristian, 2)`; `bianca \= cristian` reușește, deci soluția este X = cristian.

7.2 eligibil_licenta(emil): două negații imbricate

```
Call: eligibil_licenta(emil)
  Call: student(emil, 4)
  Exit: student(emil, 4)
  Call: promovat(emil, practica_licenta)
  Exit: promovat(emil, practica_licenta)
  Call: not(restanta(emil, _G5_1))
    Call: restanta(emil, _G5_1)
      Call: picat(emil, M_2)
      Exit: picat(emil, securitatea_datelor)
      Call: not(promovat(emil, securitatea_datelor))
        Call: promovat(emil, securitatea_datelor)
        Fail: promovat(emil, securitatea_datelor)
      Exit: not(promovat(emil, securitatea_datelor))
    Exit: restanta(emil, securitatea_datelor)
  Fail: not(restanta(emil, _G5_1))
Fail: eligibil_licenta(emil)
```

1. Primele două condiții reușesc: Emil e în anul 4 și are practica de licență.
2. Pentru `not(restanta(emil, _))`, motorul încearcă să demonstreze `restanta(emil, _)`.
3. `picat(emil, M_2)` găsește securitatea_datelor; apoi `not(promovat(emil, securitatea_datelor))`: `promovat` eșuează, deci `not` **reușește**.
4. Deci `restanta(emil, securitatea_datelor)` e demonstrată, iar `not(restanta(...))` **eșuează**.
5. Ultima condiție a regulii a eșuat, deci `eligibil_licenta(emil)` e false.

7.3 necesita(baze_de_date, P): recursivitate și redenumire

```
Call: necesita(baze_de_date, P)
  Call: prerechizit(P_1, baze_de_date)
  Exit: prerechizit(structuri_de_date, baze_de_date)
Exit: necesita(baze_de_date, structuri_de_date)
  Call: prerechizit(Q_2, baze_de_date)
  Exit: prerechizit(structuri_de_date, baze_de_date)
  Call: necesita(structuri_de_date, P_2)
    Call: prerechizit(P_3, structuri_de_date)
    Exit: prerechizit(tehnici_de_programare, structuri_de_date)
  Exit: necesita(structuri_de_date, tehnici_de_programare)
Exit: necesita(baze_de_date, tehnici_de_programare)
    Call: prerechizit(Q_4, structuri_de_date)
    Exit: prerechizit(tehnici_de_programare, structuri_de_date)
    Call: necesita(tehnici_de_programare, P_4)
      Call: prerechizit(P_5, tehnici_de_programare)
      Exit: prerechizit(programarea_calculatoarelor, tehnici_de_programare)
    Exit: necesita(tehnici_de_programare, programarea_calculatoarelor)
  Exit: necesita(structuri_de_date, programarea_calculatoarelor)
Exit: necesita(baze_de_date, programarea_calculatoarelor)
      Call: prerechizit(Q_6, tehnici_de_programare)
      Exit: prerechizit(programarea_calculatoarelor, tehnici_de_programare)
      Call: necesita(programarea_calculatoarelor, P_6)
        Call: prerechizit(P_7, programarea_calculatoarelor)
        Fail: prerechizit(P_7, programarea_calculatoarelor)
        Call: prerechizit(Q_8, programarea_calculatoarelor)
        Fail: prerechizit(Q_8, programarea_calculatoarelor)
      Fail: necesita(programarea_calculatoarelor, P_6)
```

1. Prima clauză (cazul de bază) dă prerechizitul direct: P = structuri_de_date.
2. La backtracking se încearcă a doua clauză: `prerechizit(Q_2, baze_de_date)` dă Q_2 = structuri_de_date, apoi apelul recursiv `necesita(structuri_de_date, P_2)`, cu variabile proaspete (P_2, nu P).
3. Fiecare nivel de recursivitate e mai indentat; așa apar tehnici_de_programare și programarea_calculatoarelor.
4. La `programarea_calculatoarelor` nu mai există prerechizite: ambele clauze eșuează (Fail) și căutarea se termină.

8. Cum funcționează motorul

8.1 Unificarea

Unificarea găsește substituția (legarea variabilelor) care face doi termeni identici. Rezultatul este un dicționar **nou** sau `None` (eșec); `{}` înseamnă succes fără legări.

Termenul A	Termenul B	Rezultat
`ion`	`ion`	succes, {}
`ion`	`maria`	eșec: constante diferite
`X`	`ion`	`{X = ion}`
`cunoaste(ion, X)`	`cunoaste(Y, maria)`	`{Y = ion, X = maria}`
`p(X)`	`p(X, Y)`	eșec: aritate diferită
`p(X, X)`	`p(a, b)`	eșec: X nu poate fi și a, și b
`X`	`f(X)`	eșec: occurs check (altfel termen infinit)

8.2 Backward chaining (rezoluție SLD)

1. Dacă lista de scopuri e goală, am găsit o soluție: o returnăm (`yield`).
2. Altfel luăm primul scop. Dacă e `\=` sau `not`, îl tratăm direct.
3. Altfel luăm, în ordine, clauzele din KB cu același nume și aceeași aritate (indexul din KnowledgeBase).
4. Fiecare clauză e **redenumită** (variabile proaspete), apoi capul ei se unifică cu scopul.
5. Dacă unificarea reușește, corpul clauzei se adaugă în fața scopurilor rămase și continuăm recursiv (căutare în adâncime).
6. Când o ramură eșuează, generatorul trece la următoarea clauză: acesta e **backtracking-ul**.

Generatorii Python fac backtracking-ul natural: fiecare `yield` dă o soluție, iar cererea soluției următoare reia execuția din bucla în care s-a oprit. Pentru că substituțiile nu se modifică pe loc, fiecare ramură își păstrează propria substituție.

8.3 Built-in-uri și protecții

Mecanism	Cum funcționează	De ce
`X \= Y`	reușește dacă `unify(X, Y)` eșuează	„diferit”, de exemplu în colegi
`not(G)`	reușește dacă G nu are nicio soluție	negație ca eșec, sub ipoteza lumii închise
redenumire	la fiecare folosire, X devine X_1, X_2...	apelurile recursive nu își amestecă variabilele
limită de adâncime	o ramură mai adâncă de 200 de niveluri e tăiată și se afișează un avertisment	protecție contra buclelor infinite (recursivitate la stânga)
index	clauzele grupate după (nume, aritate)	nu se încearcă clauze care sigur nu se potrivesc

9. Scenariu pentru demo-ul în fața profesorului

Durata totală a prezentării este de 10–15 minute; demo-ul durează aproximativ 4 minute. Porniți interfața web înainte (`python -m web`) și țineți un terminal cu `python main.py` deschis ca variantă de rezervă.

#	Ce faceți	Ce spuneți
1	Tab Graful prerechizitelor, click pe Sisteme încorporate	Baza de cunoștințe e planul real: 69 de materii și 63 de legături. Galben e tot lanțul dedus de regula recursivă necesita.
2	Exemple → #7 `necesita(sisteme_incorporate, P)`	În KB e scris un singur prerechizit; celelalte 4 sunt deduse prin recursivitate.
3	Exemple → #12 `poate_inscrie(cristian, M)`	Arhitectura lipsește pentru că a picat PL II. Răspunsul e dedus, nu scris.
4	Exemple → #16 și #17 (Bianca/Andrei la Grafică)	Regula cere TOATE prerechizitele: ∀ rescris ca ¬ ∃ ¬, cu negație ca eșec.
5	Exemple → #23 Rulează cu trace (`colegi(bianca, X)`)	Backward chaining: Call, Exit, Fail. La `bianca \= bianca` se vede backtracking-ul.
6	Studenți → emil → butonul de trace	Două negații imbricate: de aceea Emil nu e eligibil.
7	Graf → Colorează după cristian	O singură restanță în anul 1 blochează, prin lanț, materii din anii 3–4.
8	Interogare: scrieți o interogare cerută de profesor	Orice combinație de predicate funcționează, cu orice variabile.

Sfat: cu trace pornit, evitați interogările cu foarte multe soluții (de exemplu `poate_inscrie(S, M)`), care produc mii de pași.

10. Întrebări posibile și răspunsuri

Întrebare	Răspuns
De ce FOL și nu logica propozițională?	Propozițional ar trebui câte o propoziție pentru fiecare combinație student–materie (5 × 69 = 345). În FOL scriem o singură regulă cu variabile cuantificate universal, plus relații și recursivitate.
Ce este unificarea?	Găsirea substituției care face doi termeni identici, de exemplu `cunoaste(ion, X)` cu `cunoaste(Y, maria)` dă Y = ion, X = maria.
Ce e occurs check?	Interzice legarea X = f(X), care ar crea un termen infinit. Prolog-ul standard nu îl face, din motive de viteză; motorul nostru da.
De ce redenumiți variabilele?	Ca fiecare folosire a unei reguli să aibă variabilele ei; altfel apelurile recursive din necesita și-ar amesteca legările.
Forward sau backward chaining?	Backward: pornim de la întrebare și explorăm doar regulile relevante. Forward ar genera toate consecințele KB-ului.
Ce strategie de căutare?	În adâncime (DFS), clauzele în ordinea din fișier, scopurile de la stânga la dreapta, la fel ca Prolog.
`not` este negația din FOL?	Nu. Este negație ca eșec: `not(p)` înseamnă „p nu se poate demonstra din KB”, sub ipoteza lumii închise.
Cum ați exprimat „toate prerechizitele”?	$\forall P\ (A \rightarrow B) \equiv \neg \exists P\ (A \wedge \neg B)$: un predicat ajutător pentru „există un prerechizit nepromovat” și `not` în fața lui.
De ce un rezultat apare de două ori?	Există două demonstrații diferite (două drumuri în graf). Prolog face la fel.
Ce se întâmplă la recursivitate la stânga?	Buclă infinită la DFS. O evităm scriind cazul de bază primul și recursia la dreapta; limita de adâncime e plasa de siguranță.
Limitări?	Fără aritmetică (nu se pot aduna creditele în KB), negația nu e clasică, DFS poate fi exponențial, poate_inscrie nu verifică anul de studiu.
Ce ați îmbunătăți?	Aritmetică (`is`, `<`) pentru suma creditelor, condiție pe an la înscriere, tabling (memorare) contra recalculărilor.